Midterm Group Project - Part 3: Adding Cassandra for Real-Time Order Ingestion

Part 1: Setup & Cassandra Exploration

Task 1.1: Start the Part 3 Environment

```
marti@MartitaF MINGW64 ~/OneDrive - University of South Florida/Documents/MS AIBA/Spring 2026/Big Data/Mid-Term Project/ism6562-Midterm_project_BigQuest_Query/midterm/part0
3 (part03-marta)
$ docker compose up -d
[+] up 15/15
 ✔ Network part03_midterm_network     Created                                                                                                    0.1sss
 ✔ Volume part03_sales_shard_ne_data  Created                                                                                                    0.0sss
 ✔ Volume part03_cassandra_node1_data Created                                                                                                    0.0sss
 ✔ Volume part03_sales_shard_se_data  Created                                                                                                    0.0sss
 ✔ Volume part03_hr_data              Created                                                                                                    0.0sss
 ✔ Volume part03_cassandra_node2_data Created                                                                                                    0.0sss
 ✔ Volume part03_warehouse_data       Created                                                                                                    0.0sss
 ✔ Container midterm_cassandra_node1  Healthy                                                                                                    97.9ss
 ✔ Container midterm_warehouse_db     Healthy                                                                                                    97.9ss
 ✔ Container midterm_sales_shard_ne   Healthy                                                                                                    97.8ss
 ✔ Container midterm_hr_db            Healthy                                                                                                    97.8ss
 ✔ Container midterm_sales_shard_se   Healthy                                                                                                    97.9ss
 ✔ Container midterm_cassandra_node2  Healthy                                                                                                    193.5s
 ✔ Container midterm_pgadmin          Created                                                                                                    0.2sss
 ✔ Container midterm_etl              Created                                                                                                    0.2sss

marti@MartitaF MINGW64 ~/OneDrive - University of South Florida/Documents/MS AIBA/Spring 2026/Big Data/Mid-Term Project/ism6562-Midterm_project_BigQuest_Query/midterm/part0
3 (part03-marta)
$ docker compose ps
NAME                     IMAGE                COMMAND                  SERVICE           CREATED         STATUS                   PORTS
midterm_cassandra_node1  cassandra:4.1        "docker-entrypoint.s…"   cassandra-node1   4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:9042->9042/tcp, [::]:9042->9042/
tcp
midterm_cassandra_node2  cassandra:4.1        "docker-entrypoint.s…"   cassandra-node2   4 minutes ago   Up 3 minutes (healthy)   0.0.0.0:9043->9042/tcp, [::]:9043->9042/
tcp
midterm_etl              part03-etl-service   "./entrypoint.sh"        etl-service       4 minutes ago   Up About a minute
midterm_hr_db            postgres:16          "docker-entrypoint.s…"   hr-db             4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:5502->5432/tcp, [::]:5502->5432/
tcp
midterm_pgadmin          dpage/pgadmin4:8.13  "/entrypoint.sh"         pgadmin           4 minutes ago   Up 4 minutes             0.0.0.0:8200->80/tcp, [::]:8200->80/tcp
midterm_sales_shard_ne   postgres:16          "docker-entrypoint.s…"   sales-shard-ne    4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:5512->5432/tcp, [::]:5512->5432/
tcp
midterm_sales_shard_se   postgres:16          "docker-entrypoint.s…"   sales-shard-se    4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:5511->5432/tcp, [::]:5511->5432/
tcp
midterm_warehouse_db     postgres:16          "docker-entrypoint.s…"   warehouse-db      4 minutes ago   Up 4 minutes (healthy)   0.0.0.0:5503->5432/tcp, [::]:5503->5432/
tcp
```

Task 1.2: Explore the Cassandra Cluster

```
marti@MartitaF MINGW64 ~/OneDrive - University of South Florida/Documents/MS AIBA/Spring 2026/Big Data/Mid-
3 (part03-marta)
$ docker compose exec cassandra-node1 nodetool status
Datacenter: datacenter1
=======================
Status=Up/Down
|/ State=Normal/Leaving/Joining/Moving
--  Address     Load       Tokens  Owns (effective)  Host ID                               Rack
UN  172.18.0.8  75.21 KiB   16      100.0%            affc19b5-803a-468a-807e-139fcda0ced6  rack1
UN  172.18.0.2  109.43 KiB  16      100.0%            0722db23-a679-4fb1-8b7b-45c35c9ce823  rack1
```

```
marti@MartitaF MINGW64 ~/OneDrive - University of South Florida/Documents/MS AIBA/Spring 2026/Big Data/Mid-Term Project/ism6562-Midterm_project_
3 (part03-marta)
$ docker compose exec -T cassandra-node1 cqlsh < init/cassandra-init.cql

marti@MartitaF MINGW64 ~/OneDrive - University of South Florida/Documents/MS AIBA/Spring 2026/Big Data/Mid-Term Project/ism6562-Midterm_project_
3 (part03-marta)
$ docker compose exec cassandra-node1 cqlsh

ATTENTION: All commands will be saved to history file: /root/.cassandra/cqlsh_history
This may include sensitive information such as passwords.
To disable history, use --disable-history or set 'disabled = true' in the [history] section of cqlshrc.
See https://cassandra.apache.org/doc/latest/tools/cqlsh.html for more information.

Connected to TechRetailCluster at 127.0.0.1:9042
[cqlsh 6.1.0 | Cassandra 4.1.11 | CQL spec 3.4.6 | Native protocol v5]
Use HELP for help.
cqlsh> DESCRIBE KEYSPACES;

system       system_distributed  system_traces  system_virtual_schema
system_auth  system_schema       system_views   techretail_orders

cqlsh> USE techretail_orders;
cqlsh:techretail_orders> DESCRIBE TABLES;

orders_by_customer  orders_by_product

cqlsh:techretail_orders> DESCRIBE TABLE orders_by_customer;

CREATE TABLE techretail_orders.orders_by_customer (
    customer_id int,
    order_date date,
    order_id int,
    customer_name text,
    customer_state text,
    product_category text,
    product_id int,
    product_name text,
    quantity int,
    total_price decimal,
    unit_price decimal,
    PRIMARY KEY (customer_id, order_date, order_id)
) WITH CLUSTERING ORDER BY (order_date DESC, order_id DESC)
    AND additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
```

```
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
cqlsh:techretail_orders> DESCRIBE TABLE orders_by_product;

CREATE TABLE techretail_orders.orders_by_product (
    product_id int,
    order_date date,
    order_id int,
    customer_id int,
    customer_name text,
    quantity int,
    total_price decimal,
    PRIMARY KEY (product_id, order_date, order_id)
) WITH CLUSTERING ORDER BY (order_date DESC, order_id DESC)
    AND additional_write_policy = '99p'
    AND bloom_filter_fp_chance = 0.01
    AND caching = {'keys': 'ALL', 'rows_per_partition': 'NONE'}
    AND cdc = false
    AND comment = ''
    AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy', 'max_threshold': '32', 'min_threshold': '4'}
    AND compression = {'chunk_length_in_kb': '16', 'class': 'org.apache.cassandra.io.compress.LZ4Compressor'}
    AND memtable = 'default'
    AND crc_check_chance = 1.0
    AND default_time_to_live = 0
    AND extensions = {}
    AND gc_grace_seconds = 864000
    AND max_index_interval = 2048
    AND memtable_flush_period_in_ms = 0
    AND min_index_interval = 128
    AND read_repair = 'BLOCKING'
    AND speculative_retry = '99p';
```

Task 1.3: Compare Cassandra and PostgreSQL Table Design

Written Response (150–200 words): Compare the orders_by_customer Cassandra table to the PostgreSQL orders table from Part 1. Specifically address:

1. Why does the Cassandra table include customer_name and customer_state directly in each order row, while PostgreSQL stores this in a separate customers table with a foreign key?

   The Cassandra table includes customer_name and customer_state directly in each order row because denormalization is a good practice in Cassandra. It follows a query-first design, where tables are built to support specific queries. Since JOINs across nodes are not efficiently supported, data is duplicated so all required information can be retrieved in a single query. In contrast, PostgreSQL uses normalization and foreign keys to reduce redundancy and relies on JOINs to combine related data.

2. Why are there TWO Cassandra tables (orders_by_customer and orders_by_product) for what is essentially the same order data?

   There are two Cassandra tables with repetitive order data because each table is designed to serve a specific query. Cassandra tables are created based on how data will be accessed, and since JOINs across tables are costly, data is duplicated to allow efficient querying. The orders_by_customer table supports queries by customer, while orders_by_product supports queries by product, each including all necessary order information.

Part 2: Query-First Design Analysis

Task 2.1: Run Valid Cassandra Queries

```
cqlsh:techretail_orders> -- Query 1: All orders for customer 1 (Alice)
SELECT order_id, order_date, product_name, quantity, total_pricecqlsh:techretail_orders> SELECT order_id, order_date, product_name, quantity, total_price
                       ... FROM techretail_orders.orders_by_customer
                       ... WHERE customer_id = 1;

 order_id | order_date | product_name    | quantity | total_price
----------+------------+-----------------+----------+-------------
       25 | 2026-02-18 |       USB-C Hub |        1 |       49.99
       17 | 2026-02-03 | Ergonomic Chair |        1 |      449.99
        8 | 2026-01-18 | Monitor 27-inch |        1 |      349.99
        2 | 2026-01-05 |  Wireless Mouse |        2 |       59.98
        1 | 2026-01-05 |   Laptop Pro 15 |        1 |     1299.99

(5 rows)
cqlsh:techretail_orders> -- Query 2: All orders for product 1 (Laptop Pro 15)
cqlsh:techretail_orders> SELECT order_id, order_date, customer_name, quantity, total_price
                       ... FROM techretail_orders.orders_by_product
                       ... WHERE product_id = 1;

 order_id | order_date | customer_name  | quantity | total_price
----------+------------+----------------+----------+-------------
       23 | 2026-02-14 |      Eva Davis |        1 |     1299.99
       16 | 2026-02-01 |   Frank Miller |        1 |     1299.99
       10 | 2026-01-22 | Carol Williams |        1 |     1299.99
        1 | 2026-01-05 |  Alice Johnson |        1 |     1299.99

(4 rows)
```

Task 2.2: Attempt an Invalid Query

```
cqlsh:techretail_orders> -- This will FAIL
cqlsh:techretail_orders> SELECT * FROM techretail_orders.orders_by_customer
                       ... WHERE product_id = 1;
InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot execute this query as it might involve data filtering and thus may have unpredictable perfo
ce. If you want to execute this query despite the performance unpredictability, use ALLOW FILTERING"
```

Written Response (100–150 words): Explain why Cassandra rejects this query. How does the partition key design of orders_by_customer (partition key = customer_id) relate to how Cassandra stores and retrieves data? How does query-first design address this limitation?

Cassandra rejects this query because the orders_by_customer table is partitioned by customer_id, meaning data is physically stored based on that key. If a query filters by product_id instead, the data may be spread across multiple partitions, making it inefficient or unsupported to retrieve. Cassandra stores and retrieves data by partition key, so queries must include the partition key to efficiently locate data. Without it, Cassandra would need to scan multiple partitions, which it avoids by rejecting the query.

Query-first design addresses this limitation by creating tables tailored to specific query patterns. Instead of relying on flexible queries, Cassandra denormalizes data and creates separate tables, such as orders_by_product, to support different access patterns without requiring cross-partition operations.

Part 3: Write Orders to Cassandra

Task 3.1: Insert a New Order into Both Tables

```
cqlsh> -- Insert into orders_by_customer
cqlsh> INSERT INTO techretail_orders.orders_by_customer
   ...     (customer_id, order_date, order_id, product_id, product_name,
   ...      product_category, quantity, unit_price, total_price,
   ...      customer_name, customer_state)
   ... VALUES (2, '2026-02-25', 28, 1, 'Laptop Pro 15', 'Electronics',
   ...         1, 1299.99, 1299.99, 'Bob Smith', 'FL');
cqlsh> -- Insert into orders_by_product
cqlsh> INSERT INTO techretail_orders.orders_by_product
   ...     (product_id, order_date, order_id, customer_id, customer_name,
   ...      quantity, total_price)
   ... VALUES (1, '2026-02-25', 28, 2, 'Bob Smith', 1, 1299.99);
cqlsh> SELECT * FROM techretail_orders.orders_by_customer
   ... WHERE customer_id = 2 AND order_date = '2026-02-25';

 customer_id | order_date | order_id | customer_name | customer_state | product_category | product_id | product_name  | quantity | total_price | unit_price
-------------+------------+----------+---------------+----------------+------------------+------------+---------------+----------+-------------+------------
           2 | 2026-02-25 |       28 |     Bob Smith |             FL |      Electronics |          1 | Laptop Pro 15 |        1 |     1299.99 |    1299.99

(1 rows)
cqlsh> SELECT * FROM techretail_orders.orders_by_product
   ... WHERE product_id = 1 AND order_date = '2026-02-25';

 product_id | order_date | order_id | customer_id | customer_name | quantity | total_price
------------+------------+----------+-------------+---------------+----------+-------------
          1 | 2026-02-25 |       28 |           2 |     Bob Smith |        1 |     1299.99

(1 rows)
```

Task 3.2: Dual-Write Consistency Challenge

Written Response (100–150 words): What happens if one INSERT succeeds and the other fails? For example, the order appears in orders_by_customer but not in orders_by_product. How would you handle this in production? Connect your answer to Cassandra's **logged batch** statements (covered in Week 5).

If one INSERT succeeds and the other fails, it creates data inconsistency because the order would exist in one table but not in the other. This can lead to incomplete query results depending on which table is queried. Simply retrying the failed insert could also introduce issues if not

handled carefully. In production, this can be addressed using Cassandra's logged batch statements. A logged batch ensures that all updates are written to a batch log before being applied, allowing Cassandra to retry failed operations and maintain consistency across tables. While it does not provide full ACID guarantees, it helps ensure that either all updates are eventually applied or the system can recover and complete them, reducing the risk of partial writes.

Part 4: Update ETL Pipeline

Task 4.1: Analyze the Current ETL Script

Written Response (100–150 words): The current ETL extracts orders from the PostgreSQL shards. You need to change it to extract orders from Cassandra instead. Identify what needs to change:

- What new Python import is needed? (from cassandra.cluster import Cluster)

  The new ETL needs to extract orders from Cassandra instead of PostgreSQL shards, therefore we need: from cassandra.cluster import Cluster.

- What new function would you write to extract orders from Cassandra?

  To extract orders from Cassandra I would write the new function: extract_orders_from_cassandra(). This function would query the appropriate keyspace and retrieve order data.

- Which existing lines in run_etl() need to change?

  These are the lines that would need to change since they are pulling orders from PostgreSQL.

```
# Extract from SE shard
  se_conn = get_connection("SALES_SHARD_SE")
  se_customers, se_products, se_orders = extract_sales_from_shard(se_conn, "sales-shard-se")
  se_conn.close()

  # Extract from NE shard
  ne_conn = get_connection("SALES_SHARD_NE")
  ne_customers, ne_products, ne_orders = extract_sales_from_shard(ne_conn, "sales-shard-ne")
  ne_conn.close()
```

```
# Merge shard results
customers = se_customers + ne_customers
products = se_products  # Products are identical on both shards (reference table)
orders = se_orders + ne_orders
```

- What stays the same? (Customer and product extraction from PostgreSQL, all transform and load phases)

  Customer and product extraction from the PostgreSQL shards remains unchanged, including the connections and use of extract_sales_from_shard(). The extraction of employee data from the HR database also stays the same. Additionally, all transformation steps and the loading process into the data warehouse remain unchanged.

Task 4.2: Modify the ETL to Extract from Cassandra

1. Add the Cassandra import at the top of the file:

```
import os
import sys
from datetime import date, timedelta

from cassandra.cluster import Cluster
import psycopg2
from psycopg2.extras import execute_values
```

2. Add a new function to extract orders from Cassandra:

```
def extract_orders_from_cassandra():
    """Extract orders from Cassandra's orders_by_customer table."""
    hosts = os.environ["CASSANDRA_HOSTS"].split(",")
    keyspace = os.environ["CASSANDRA_KEYSPACE"]
    cluster = Cluster(hosts)
    session = cluster.connect(keyspace)

    # Query all orders — need to get all customer_ids first
    # For simplicity, query with a full table scan (small dataset)
    rows = session.execute("SELECT order_id, customer_id, product_id, quantity, total_price,
order_date FROM orders_by_customer")
```

```
    # Deduplicate by order_id (in case of overlapping partitions)
    seen = set()
    orders = []
    for row in rows:
        if row.order_id not in seen:
            seen.add(row.order_id)
            orders.append((
                row.order_id,
                row.customer_id,
                row.product_id,
                row.quantity,
                float(row.total_price),
                row.order_date.date()
            ))

    print(f"  Extracted {len(orders)} orders from Cassandra")
    cluster.shutdown()
    return orders
```

3. **In** run_etl()**, replace the order extraction from PostgreSQL shards with the Cassandra function.** Keep customer and product extraction from PostgreSQL.

```
def run_etl():
    """Execute the full ETL pipeline."""
    print("=" * 60)
    print("ETL Pipeline Starting")
    print("=" * 60)

    # --- EXTRACT ---
    print("\n[EXTRACT] Reading from operational databases...")

    # Extract from SE shard
    se_conn = get_connection("SALES_SHARD_SE")
    se_customers, se_products, _ = extract_sales_from_shard(se_conn, "sales-shard-se")
    se_conn.close()

    # Extract from NE shard
    ne_conn = get_connection("SALES_SHARD_NE")
    ne_customers, ne_products, _ = extract_sales_from_shard(ne_conn, "sales-shard-ne")
    ne_conn.close()

    # Merge shard results
    customers = se_customers + ne_customers
```

```
products = se_products  # Products are identical on both shards (reference table)
orders = extract_orders_from_cassandra()
print(f"  Merged: {len(customers)} customers, {len(products)} products, {len(orders)} orders")

hr_conn = get_connection("HR")
employees = extract_hr(hr_conn)
hr_conn.close()

# --- TRANSFORM ---
print("\n[TRANSFORM] Preparing dimension data...")
dim_date_rows = generate_dim_date_rows(2026)
print("  Employee dimension: denormalized with department info (done in extract query)")

# --- LOAD ---
print("\n[LOAD] Writing to data warehouse...")
wh_conn = get_connection("WAREHOUSE")
cur = wh_conn.cursor()

load_dim_date(cur, dim_date_rows)
load_dim_customer(cur, customers)
load_dim_product(cur, products)
load_dim_employee(cur, employees)
load_fact_sales(cur, orders)

wh_conn.commit()
cur.close()
wh_conn.close()

print("\n" + "=" * 60)
print("ETL Pipeline Complete")
print("=" * 60)
```

Modified ETL code:

```python
from cassandra.cluster import Cluster
import psycopg2
from psycopg2.extras import execute_values

# ----------------------------------------------------------------------------
# Cassandra extraction function
# ----------------------------------------------------------------------------

def extract_orders_from_cassandra():
    """Extract orders from Cassandra's orders_by_customer table."""
    hosts = os.environ["CASSANDRA_HOSTS"].split(",")
    keyspace = os.environ["CASSANDRA_KEYSPACE"]
    cluster = Cluster(hosts)
    session = cluster.connect(keyspace)

    # Query all orders - need to get all customer_ids first
    # For simplicity, query with a full table scan (small dataset)
    rows = session.execute("SELECT order_id, customer_id, product_id, quantity, total_price, order_date FROM orders_by_customer")

    # Deduplicate by order_id (in case of overlapping partitions)
    seen = set()
    orders = []
    for row in rows:
        if row.order_id not in seen:
            seen.add(row.order_id)
            orders.append((
                row.order_id,
                row.customer_id,
                row.product_id,
                row.quantity,
                float(row.total_price),
                row.order_date.date()
            ))

    print(f"  Extracted {len(orders)} orders from Cassandra")
    cluster.shutdown()
    return orders
```

```python
def run_etl():
    """Execute the full ETL pipeline."""
    print("=" * 60)
    print("ETL Pipeline Starting")
    print("=" * 60)

    # --- EXTRACT ---
    print("\n[EXTRACT] Reading from operational databases...")

    # Extract from SE shard
    se_conn = get_connection("SALES_SHARD_SE")
    se_customers, se_products, _ = extract_sales_from_shard(se_conn, "sales-shard-se")
    se_conn.close()

    # Extract from NE shard
    ne_conn = get_connection("SALES_SHARD_NE")
    ne_customers, ne_products, _ = extract_sales_from_shard(ne_conn, "sales-shard-ne")
    ne_conn.close()

    # Merge shard results
    customers = se_customers + ne_customers
    products = se_products  # Products are identical on both shards (reference table)
    orders = extract_orders_from_cassandra()
    print(f"  Merged: {len(customers)} customers, {len(products)} products, {len(orders)} orders")

    hr_conn = get_connection("HR")
    employees = extract_hr(hr_conn)
    hr_conn.close()

    # --- TRANSFORM ---
    print("\n[TRANSFORM] Preparing dimension data...")
    dim_date_rows = generate_dim_date_rows(2026)
    print("  Employee dimension: denormalized with department info (done in extract query)")
```

Successful output:

```
marti@MartitaF MINGW64 ~/OneDrive - University of South Florida/Documents/MS AIBA/Spring 2026/Big Data/Mid-T
3 (part03-marta)
$ docker compose logs -f etl-service
midterm_etl   | === ETL Service Starting ===
midterm_etl   | Seeding Cassandra...
midterm_etl   | ./entrypoint.sh: line 11: /app/cassandra-seed.sh: cannot execute: required file not found
midterm_etl   |
midterm_etl   | Running initial ETL load...
midterm_etl   | ============================================================
midterm_etl   | ETL Pipeline Starting
midterm_etl   | ============================================================
midterm_etl   |
midterm_etl   | [EXTRACT] Reading from operational databases...
midterm_etl   |   Extracted 7 customers from sales-shard-se
midterm_etl   |   Extracted 10 products from sales-shard-se
midterm_etl   |   Extracted 24 orders from sales-shard-se
midterm_etl   |   Extracted 1 customers from sales-shard-ne
midterm_etl   |   Extracted 10 products from sales-shard-ne
midterm_etl   |   Extracted 3 orders from sales-shard-ne
midterm_etl   |   Extracted 28 orders from Cassandra
midterm_etl   |   Merged: 8 customers, 10 products, 28 orders
midterm_etl   |   Extracted 12 employees from hr-db
midterm_etl   |
midterm_etl   | [TRANSFORM] Preparing dimension data...
midterm_etl   |   Generated 365 dim_date rows for 2026
midterm_etl   |   Employee dimension: denormalized with department info (done in extract query)
midterm_etl   |
midterm_etl   | [LOAD] Writing to data warehouse...
midterm_etl   |   Loaded dim_date (365 rows, conflicts skipped)
midterm_etl   |   Loaded dim_customer (8 rows, upserted)
midterm_etl   |   Loaded dim_product (10 rows, upserted)
midterm_etl   |   Loaded dim_employee (12 rows, upserted)
midterm_etl   |   Loaded fact_sales (1 inserted, 27 already existed)
midterm_etl   |
midterm_etl   | ============================================================
midterm_etl   | ETL Pipeline Complete
midterm_etl   | ============================================================
midterm_etl   |
midterm_etl   | Starting cron for scheduled runs (every 5 minutes)...
```

Warehouse verification:

```
marti@MartitaF MINGW64 ~/OneDrive - University of South Florida/Documents/MS AIBA/Spring 2026/B
3 (part03-marta)
$ docker compose exec warehouse-db psql -U student -d warehouse
psql (16.12 (Debian 16.12-1.pgdg13+1))
Type "help" for help.

warehouse=# SELECT f.sale_key, d.full_date, c.first_name, c.last_name,
       p.product_name, f.quantity, f.total_price
FROM fact_sales f
JOIN dim_date d ON f.date_key = d.date_key
JOIN dim_customer c ON f.customer_key = c.customer_key
JOIN dim_product p ON f.product_key = p.product_key
ORDER BY f.sale_key DESC
LIMIT 3;
 sale_key | full_date  | first_name | last_name | product_name   | quantity | total_price
----------+------------+------------+-----------+----------------+----------+-------------
       38 | 2026-02-25 | Bob        | Smith     | Laptop Pro 15  |        1 |     1299.99
       27 | 2026-02-14 | Eva        | Davis     | Laptop Pro 15  |        1 |     1299.99
       26 | 2026-01-28 | Eva        | Davis     | Standing Desk  |        1 |      599.99
(3 rows)
```

Part 5: Polyglot Persistence Reflection

Task 5.1: Compare the Three Database Roles

Create a comparison table and written response.

Aspect	PostgreSQL (Shards)	Cassandra (Ingestion)	PostgreSQL (Warehouse)
Primary role	Transactional system (OLTP) storing operational data across shards	High-throughput ingestion layer for scalable writes	Analytics for reporting and querying (OLAP)
CAP positioning	CA	AP	CA
Consistency model	Strong consistency (ACID transactions)	Tunable consistency (eventual consistency by default)	Strong consistency (ACID, reliable reporting)
Best for	Structured transactions, joins, normalized data	High write throughput, distributed data, scalability	Complex queries, aggregations, business intelligence
Limitation	Hard to scale horizontally, expensive joins across shards	Limited joins, data duplication, eventual consistency	Not optimized for high-frequency writes, slower ingestion, only vertical scaling

Written Response (200–250 words): Explain how each database technology in this architecture serves a different purpose and why a single database technology cannot efficiently handle all three roles.

Each database technology in this architecture serves a different purpose based on its strengths. PostgreSQL shards are best suited for ACID transactions and reference data, as they efficiently handle structured transactions and normalized data. However, for high-velocity write workloads, Cassandra is the better choice because it provides high write throughput, scalability, and distributed data handling. Finally, for analytical queries on a star schema, the PostgreSQL data warehouse is the most appropriate option, as it can efficiently support complex queries, joins, and relationships.

Task 5.2: Argue For or Against Cassandra as a Warehouse (2 pts)

Written Response (200–250 words): Argue for or against replacing the PostgreSQL warehouse with Cassandra for analytical queries. Consider the following:

1. Can Cassandra support star schema JOINs between fact and dimension tables?

   Cassandra does not support star schema JOINs between fact and dimension tables because it follows a query-first design. Since Cassandra distributes data across partitions, performing joins across partitions is inefficient and not supported in the same way as relational databases. Instead, data is denormalized and tables are designed to match query patterns.

2. How does Cassandra handle GROUP BY aggregations compared to PostgreSQL?

   Cassandra only allows grouping on columns that are part of the partition key and does not support aggregations across large datasets or multiple partitions efficiently. In many cases, queries require the use of ALLOW FILTERING, which can lead to full table scans and poor performance. In contrast, PostgreSQL fully supports GROUP BY operations, enabling efficient aggregations, complex queries, and analytical functions across entire tables.

3. What consistency guarantees do analytical workloads typically require?

   Analytical workloads typically require strong consistency to ensure that query results are accurate, complete, and reliable. Since these workloads are used for reporting and decision-making, they rely on consistent and stable data rather than eventually consistent data that may be temporarily outdated or incomplete.

4. Is Cassandra's query-first design compatible with ad-hoc analytical queries?

   Cassandra's query-first design is not compatible with ad-hoc analytical queries because tables must be designed in advance based on known query patterns.

Expected conclusion: Cassandra is a poor fit for the warehouse role because (1) it does not support JOINs needed for star schema queries, (2) GROUP BY aggregations require ALLOW FILTERING and are inefficient, (3) analytical workloads need strong consistency for accurate reporting, and (4) the query-first design requires knowing all queries in advance, which conflicts with ad-hoc analytics.